

Assignment On
A Comprehensive Approach to Vehicle Type Classification Using
Convolutional Neural Networks (CNN)



Department of Computer Science & Engineering
CSE 5104 – Digital Image Processing (DIP)
Semester: Summer-2025, Section: 1 (One)

Submitted to
Dr. Md. Zulfiker Mahmud
Professor

Submitted by
Md. Arafat Hossain Siam
ID: M250205007

Department of Computer Science & Engineering
Jagannath University, Dhaka-1100, Bangladesh.

Appendix

Load Zip Dataset

```
[1]
✓ 2h
from google.colab import files

uploaded = files.upload()
```

Choose Files Vehicles-Dataset.zip
Vehicles-Dataset.zip(application/x-zip-compressed) - 792875720 bytes, last modified: 11/20/2025 - 100% done
Saving Vehicles-Dataset.zip to Vehicles-Dataset.zip

Extracting Zip File

```
[2]
✓ 10s
import zipfile

zip_ref = zipfile.ZipFile('/content/Vehicles-Dataset.zip', 'r')
zip_ref.extractall('/content')
zip_ref.close()
```

```
[3]
✓ 8s
!pip install split-folders

Collecting split-folders
  Downloading split_folders-0.5.1-py3-none-any.whl.metadata (6.2 kB)
  Downloading split_folders-0.5.1-py3-none-any.whl (8.4 kB)
Installing collected packages: split-folders
Successfully installed split-folders-0.5.1
```

```
[4]
✓ 8s
import splitfolders

splitfolders.ratio("/content/Vehicles-Dataset", output="/content/vehicles_dataset_split", seed=42, ratio=(.8, .2))
```

Copying files: 3200 files [00:08, 388.71 files/s]

Preparing Dataset

```
[5]
✓ 22s
from tensorflow import keras
import tensorflow as tf

# generators
train_ds = keras.utils.image_dataset_from_directory(
    directory = '/content/vehicles_dataset_split/train',
    labels = 'inferred',
    label_mode = 'int',
    batch_size = 32,
    image_size = (128,128),
    color_mode='rgb',
    shuffle=True
)
```

```
val_ds = keras.utils.image_dataset_from_directory(
    directory = '/content/vehicles_dataset_split/val',
    labels = 'inferred',
    label_mode = 'int',
    batch_size = 32,
    image_size = (128,128),
    color_mode='rgb',
    shuffle=True
)
```

Found 2558 files belonging to 4 classes.
Found 640 files belonging to 4 classes.

Image Visualization

[6]
✓ 1s

```
import matplotlib.pyplot as plt
import numpy as np

# Get class names
class_names = train_ds.class_names

# Extract one batch from the dataset
for images, labels in train_ds.take(1):
    images = images.numpy().astype("uint8")
    labels = labels.numpy()

# Set up subplot grid
plt.figure(figsize=(10, 10))

for i in range(9): # First 9 images
    ax = plt.subplot(3, 3, i + 1)
    plt.imshow(images[i])
    plt.title(class_names[labels[i]])
    plt.axis("off")
```

plt.tight_layout()
plt.show()

...

Ships



Ships



Planes



Planes



Bus



Bus



Planes



Motorcycles



Planes



Model Building

[7]

✓ 0s

```
from keras import Sequential
from keras.layers import Dense, Conv2D, MaxPooling2D, Flatten, BatchNormalization, Input, Dropout

model = Sequential()
model.add(Input(shape=(128, 128, 3)))

model.add(Conv2D(32, kernel_size=(3, 3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(3, 3), padding='same'))

model.add(Conv2D(64, kernel_size=(3, 3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(3, 3), padding='same'))
```

```
[7]
✓ 0s model.add(Conv2D(128, kernel_size=(3, 3),activation='relu',padding="same"))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(3, 3),padding='same'))

model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(4, activation='softmax'))
```

```
[8]
✓ 0s model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 128, 128, 32)	896
batch_normalization (BatchNormalization)	(None, 128, 128, 32)	128
max_pooling2d (MaxPooling2D)	(None, 43, 43, 32)	0
conv2d_1 (Conv2D)	(None, 43, 43, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 43, 43, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 15, 15, 64)	0
conv2d_2 (Conv2D)	(None, 15, 15, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 15, 15, 128)	512

max_pooling2d_2 (MaxPooling2D)	(None, 5, 5, 128)	0
flatten (Flatten)	(None, 3200)	0
dense (Dense)	(None, 128)	409,728
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 4)	260

Total params: 512,388 (1.95 MB)
Trainable params: 511,940 (1.95 MB)
Non-trainable params: 448 (1.75 KB)

```
[9]
0s model.compile(optimizer='Adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

Model Training With Time

```
[10]
✓ 42m import time
start_time = time.time()

epochs=20
history = model.fit(train_ds, epochs=epochs, validation_data=val_ds)

end_time = time.time()
print(f"\nTotal Training Time: {end_time - start_time:.2f} seconds")
```

Epoch 1/20
80/80 ————— 120s 1s/step - accuracy: 0.4303 - loss: 2.1252 - val_accuracy: 0.6672 - val_loss: 0.8945
Epoch 2/20
80/80 ————— 107s 1s/step - accuracy: 0.6272 - loss: 0.9924 - val_accuracy: 0.7688 - val_loss: 0.6294
Epoch 3/20
80/80 ————— 120s 1s/step - accuracy: 0.7561 - loss: 0.6940 - val_accuracy: 0.8391 - val_loss: 0.4599
Epoch 4/20
80/80 ————— 111s 1s/step - accuracy: 0.8108 - loss: 0.5391 - val_accuracy: 0.7937 - val_loss: 0.5981
Epoch 5/20
80/80 ————— 141s 1s/step - accuracy: 0.8633 - loss: 0.4701 - val_accuracy: 0.8547 - val_loss: 0.4041

Epoch 6/20
80/80 ————— 111s 1s/step - accuracy: 0.8595 - loss: 0.4495 - val_accuracy: 0.8344 - val_loss: 0.5763
Epoch 7/20
80/80 ————— 143s 1s/step - accuracy: 0.8915 - loss: 0.3513 - val_accuracy: 0.8094 - val_loss: 0.7292
Epoch 8/20
80/80 ————— 155s 2s/step - accuracy: 0.9000 - loss: 0.3242 - val_accuracy: 0.8438 - val_loss: 0.5304
Epoch 9/20
80/80 ————— 110s 1s/step - accuracy: 0.9118 - loss: 0.2740 - val_accuracy: 0.6594 - val_loss: 1.4061
Epoch 10/20
80/80 ————— 141s 1s/step - accuracy: 0.9103 - loss: 0.2998 - val_accuracy: 0.7766 - val_loss: 0.6519
Epoch 11/20
80/80 ————— 110s 1s/step - accuracy: 0.9363 - loss: 0.2150 - val_accuracy: 0.9000 - val_loss: 0.3506
Epoch 12/20
80/80 ————— 141s 1s/step - accuracy: 0.9313 - loss: 0.2354 - val_accuracy: 0.6187 - val_loss: 2.4464
Epoch 13/20
80/80 ————— 155s 2s/step - accuracy: 0.9364 - loss: 0.2027 - val_accuracy: 0.9047 - val_loss: 0.3842
Epoch 14/20
80/80 ————— 133s 1s/step - accuracy: 0.9443 - loss: 0.1926 - val_accuracy: 0.8797 - val_loss: 0.3639
Epoch 15/20
80/80 ————— 141s 1s/step - accuracy: 0.9614 - loss: 0.1255 - val_accuracy: 0.8734 - val_loss: 0.4920
Epoch 16/20
80/80 ————— 110s 1s/step - accuracy: 0.9462 - loss: 0.1842 - val_accuracy: 0.7875 - val_loss: 0.7171
Epoch 17/20
80/80 ————— 150s 1s/step - accuracy: 0.9604 - loss: 0.1302 - val_accuracy: 0.8687 - val_loss: 0.3880

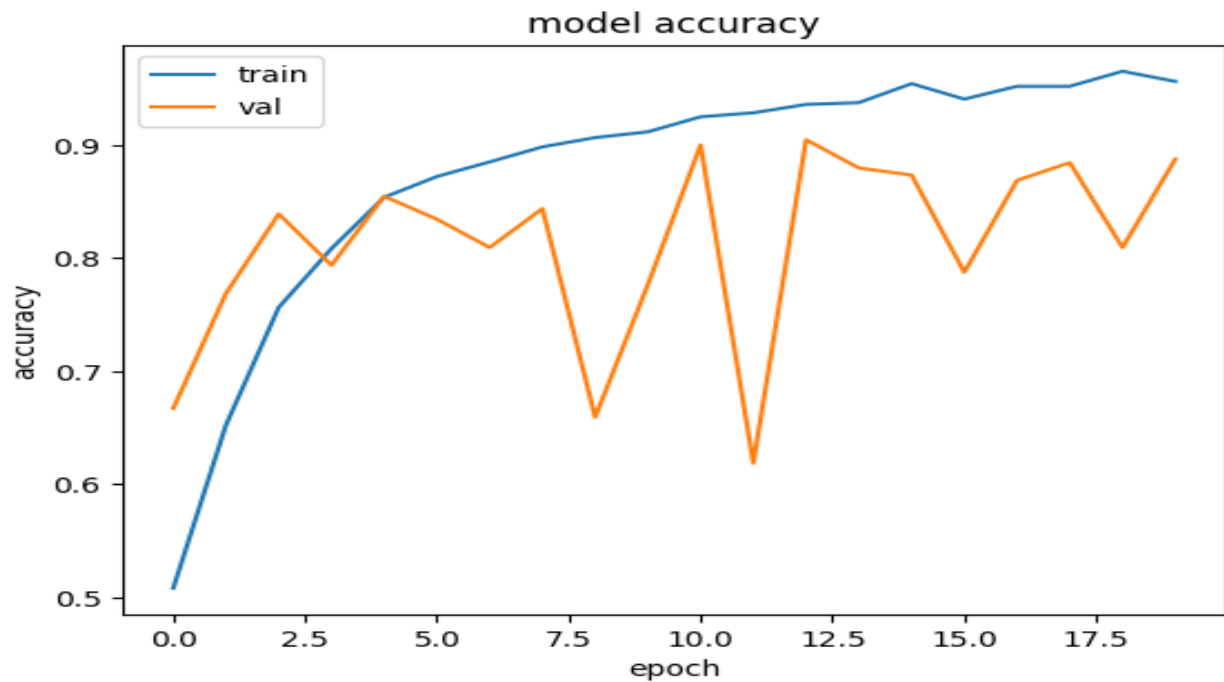
Epoch 18/20
80/80 ————— 111s 1s/step - accuracy: 0.9548 - loss: 0.1443 - val_accuracy: 0.8844 - val_loss: 0.4104
Epoch 19/20
80/80 ————— 142s 1s/step - accuracy: 0.9600 - loss: 0.1225 - val_accuracy: 0.8094 - val_loss: 0.8190
Epoch 20/20
80/80 ————— 112s 1s/step - accuracy: 0.9606 - loss: 0.1085 - val_accuracy: 0.8875 - val_loss: 0.3316

Total Training Time: 2562.97 seconds

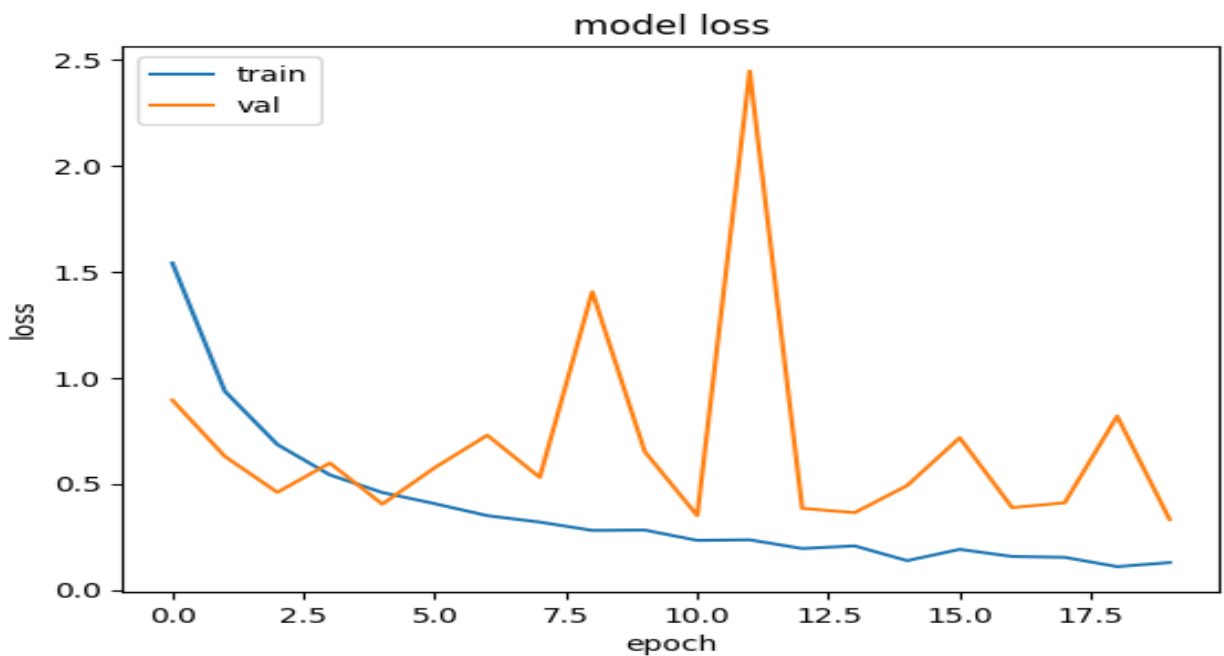
Plot Accuracy and Loss

```
import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'val'], loc='upper left')
plt.show()
```



```
[12] ✓ 0s plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'val'], loc='upper left')
plt.show()
```



✓ Validation Accuracy

[13]
✓ 8s

```
val_loss, val_accuracy = model.evaluate(val_ds)

print(f"Validation Accuracy: {val_accuracy:.4f}")
```

✓

20/20 ————— 9s 414ms/step - accuracy: 0.8800 - loss: 0.3535
Validation Accuracy: 0.8875

✓ Confusion Matrix & Classification Report

[14]
✓ 15s

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report

# Get class names
class_names = val_ds.class_names
```

[14]
✓ 15s

```
# Step 1: Collect true and predicted labels
y_true = []
y_pred = []

for images, labels in val_ds:
    preds = model.predict(images)
    preds = np.argmax(preds, axis=1) # get class with highest probability

    y_true.extend(labels.numpy())
    y_pred.extend(preds)

y_true = np.array(y_true)
y_pred = np.array(y_pred)
```

✓

1/1 ————— 1s 650ms/step
1/1 ————— 0s 249ms/step
1/1 ————— 0s 342ms/step
1/1 ————— 0s 358ms/step
1/1 ————— 0s 378ms/step
1/1 ————— 1s 921ms/step
1/1 ————— 1s 811ms/step
1/1 ————— 0s 428ms/step
1/1 ————— 0s 248ms/step


```

1/1 _____ 0s 300ms/step
1/1 _____ 0s 439ms/step
1/1 _____ 1s 500ms/step
1/1 _____ 1s 534ms/step
1/1 _____ 0s 499ms/step
1/1 _____ 0s 418ms/step
1/1 _____ 0s 368ms/step
1/1 _____ 1s 542ms/step
1/1 _____ 0s 375ms/step
1/1 _____ 0s 241ms/step
1/1 _____ 0s 222ms/step

```

```

# Step 2: Print classification report
print("Classification Report:")
print(classification_report(y_true, y_pred, target_names=class_names))

```

```

Classification Report:

```

	precision	recall	f1-score	support
Bus	0.91	0.84	0.87	160
Motorcycles	0.92	0.96	0.94	160
Planes	0.82	0.96	0.89	160
Ships	0.93	0.78	0.85	160

▼

accuracy			0.89	640
macro avg	0.89	0.89	0.89	640
weighted avg	0.89	0.89	0.89	640

[16]

✓ 0s

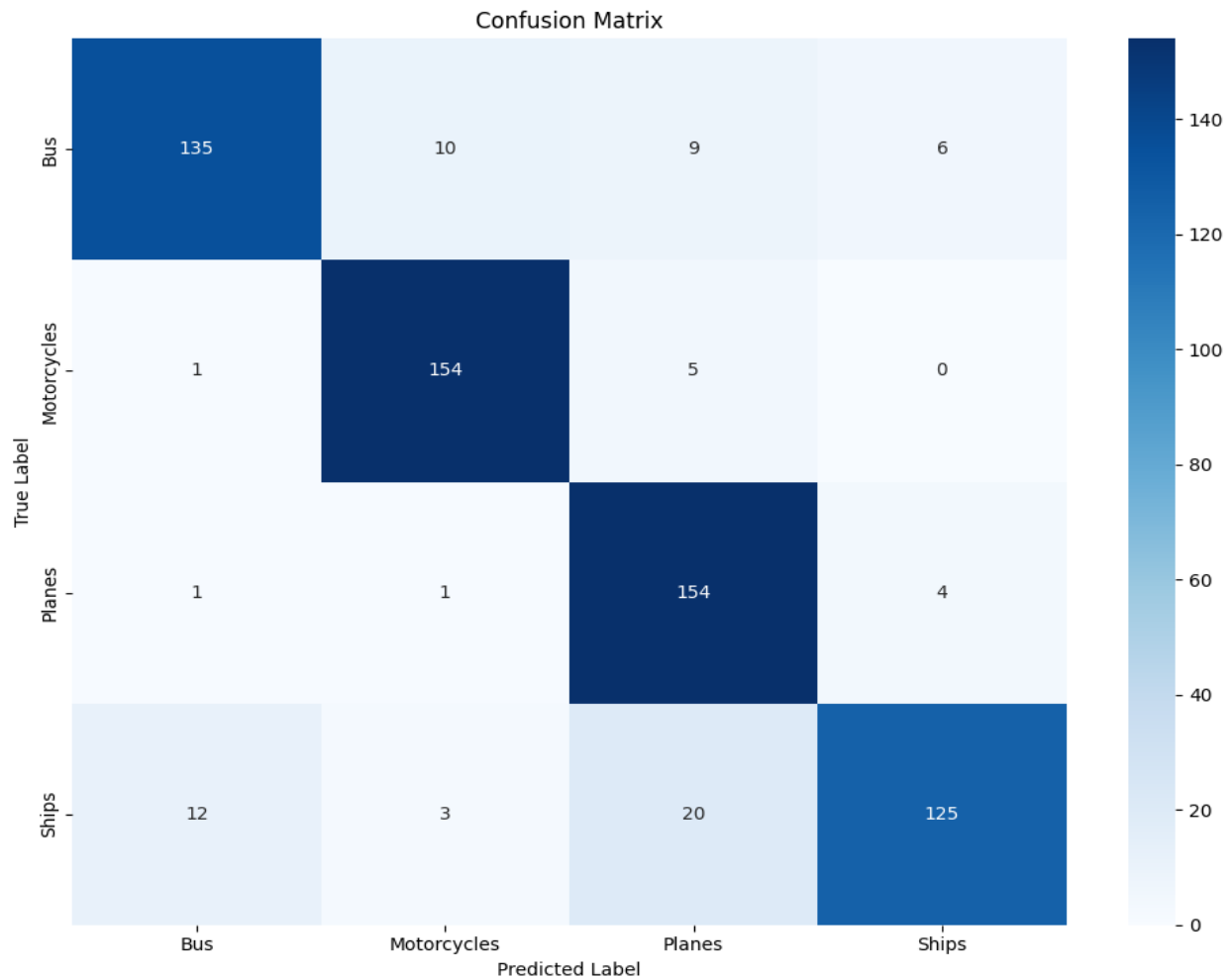


```

# Step 3: Confusion matrix
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=class_names, yticklabels=class_names)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.tight_layout()
plt.show()

```



Sample Predictions on Validation Images

[17]
✓ 12s

```
plt.figure(figsize=(12, 12))

for images, labels in val_ds.take(1):
    preds = np.argmax(model.predict(images), axis=1)

    for i in range(9):
        plt.subplot(3, 3, i+1)
        plt.imshow(images[i].numpy().astype("uint8"))
        true_label = class_names[labels[i]]
        pred_label = class_names[preds[i]]
        plt.title(f"True: {true_label}\nPred: {pred_label}")
        plt.axis("off")

plt.tight_layout()
plt.show()
```

True: Planes
Pred: Planes



True: Bus
Pred: Bus



True: Ships
Pred: Ships



True: Ships
Pred: Ships



True: Planes
Pred: Planes



True: Bus
Pred: Bus



True: Ships
Pred: Ships



True: Motorcycles
Pred: Motorcycles



True: Planes
Pred: Planes



▼ Data Augmentation Visualization

[18]
✓ 1s

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)

sample_img = images[0].numpy().astype("uint8")

plt.figure(figsize=(10,4))
for i in range(5):
    aug = datagen.random_transform(sample_img)
    plt.subplot(1,5,i+1)
    plt.imshow(aug)
    plt.axis("off")
plt.show()
```



Per-Class Accuracy

[19]
✓ 12s

```
class_correct = {name:0 for name in class_names}
class_total = {name:0 for name in class_names}

for images, labels in val_ds:
    preds = np.argmax(model.predict(images), axis=1)
    for label, pred in zip(labels, preds):
        class_total[class_names[label]] += 1
        if label == pred:
            class_correct[class_names[label]] += 1

print("\nPer-Class Accuracy:")
for cls in class_names:
    acc = class_correct[cls] / class_total[cls]
    print(f"{cls}: {acc*100:.2f}%")
```

▼

```
1/1 ————— 0s 420ms/step
1/1 ————— 0s 233ms/step
1/1 ————— 0s 254ms/step
1/1 ————— 0s 455ms/step
```

```
1/1 ————— 0s 457ms/step
1/1 ————— 0s 457ms/step
... 1/1 ————— 0s 440ms/step
1/1 ————— 0s 296ms/step
1/1 ————— 0s 248ms/step
1/1 ————— 0s 356ms/step
1/1 ————— 0s 459ms/step
1/1 ————— 1s 562ms/step
1/1 ————— 1s 525ms/step
1/1 ————— 0s 453ms/step
1/1 ————— 0s 238ms/step
1/1 ————— 0s 323ms/step
1/1 ————— 0s 373ms/step
1/1 ————— 0s 218ms/step
1/1 ————— 0s 214ms/step
1/1 ————— 0s 231ms/step
```

```
Per-Class Accuracy:
Bus: 84.38%
Motorcycles: 96.25%
Planes: 96.25%
Ships: 78.12%
```

▼ ROC Curve For Each Class

[20]

✓ 9s

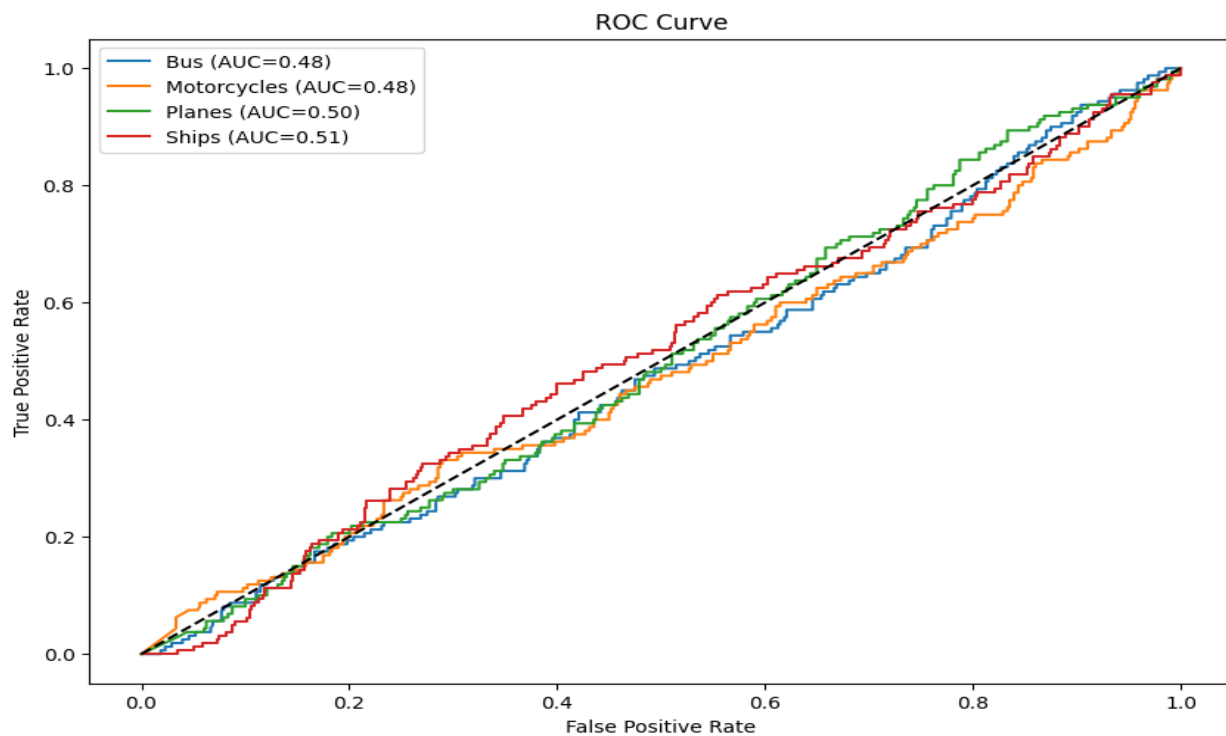
```
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc

y_true_bin = label_binarize(y_true, classes=[0,1,2,3])
y_pred_prob = model.predict(val_ds)

plt.figure(figsize=(10,7))

for i, class_name in enumerate(class_names):
    fpr, tpr, _ = roc_curve(y_true_bin[:,i], y_pred_prob[:,i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f"{class_name} (AUC={roc_auc:.2f})")

plt.plot([0,1],[0,1], 'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```



Gradio Web App

```
pip install gradio
```

```
Requirement already satisfied: gradio in /usr/local/lib/python3.12/dist-packages (5.49.1)
Requirement already satisfied: aiofiles<25.0,>=22.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (24.1.0)
Requirement already satisfied: anyio<5.0,>=3.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (4.11.0)
Requirement already satisfied: brotli<1.0,>=0.115.2 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.121.1)
Requirement already satisfied: fastapi<1.0,>=0.115.2 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.121.1)
Requirement already satisfied: ffmpeg in /usr/local/lib/python3.12/dist-packages (from gradio) (1.0.0)
Requirement already satisfied: gradio-client==1.13.3 in /usr/local/lib/python3.12/dist-packages (from gradio) (1.13.3)
Requirement already satisfied: groovy~=0.1 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.1.2)
Requirement already satisfied: httpx<1.0,>=0.24.1 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.28.1)
Requirement already satisfied: huggingface-hub<2.0,>=0.33.5 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.36.0)
Requirement already satisfied: jinja2<4.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (3.1.6)
Requirement already satisfied: markupsafe<4.0,>=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (3.0.3)
Requirement already satisfied: numpy<3.0,>=1.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.0.2)
Requirement already satisfied: orjson~=3.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (3.11.4)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from gradio) (25.0)
Requirement already satisfied: pandas<3.0,>=1.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.2.2)
Requirement already satisfied: pillow<12.0,>=8.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (11.3.0)
Requirement already satisfied: pydantic<2.12,>=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.11.10)
```

```
Requirement already satisfied: pydub in /usr/local/lib/python3.12/dist-packages (from gradio) (0.25.1)
Requirement already satisfied: python-multipart>=0.0.18 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.0.20)
Requirement already satisfied: pyyaml<7.0,>=5.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (6.0.3)
Requirement already satisfied: ruff>=0.9.3 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.14.4)
Requirement already satisfied: safehttpx<0.2.0,>=0.1.6 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.1.7)
Requirement already satisfied: semantic-version~=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.10.0)
Requirement already satisfied: starlette<1.0,>=0.40.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.49.3)
Requirement already satisfied: tomlkit<0.14.0,>=0.12.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.13.3)
Requirement already satisfied: typer<1.0,>=0.12 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.20.0)
Requirement already satisfied: typing-extensions~=4.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (4.15.0)
Requirement already satisfied: uvicorn>=0.14.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.38.0)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from gradio-client==1.13.3->gradio) (2025.3.0)
Requirement already satisfied: websockets<16.0,>=13.0 in /usr/local/lib/python3.12/dist-packages (from gradio-client==1.13.3->gradio) (15.0)
Requirement already satisfied: idna=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5.0,>=3.0->gradio) (3.11)
Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/python3.12/dist-packages (from anyio<5.0,>=3.0->gradio) (1.3.1)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from fastapi<1.0,>=0.115.2->gradio) (0.0.4)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1.0,>=0.24.1->gradio) (2025.10.5)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1.0,>=0.24.1->gradio) (1.0.9)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1.0,>=0.24.1->gradio) (0.16.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.33.5->gradio) (3.20.0)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.33.5->gradio) (2.32.4)
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.33.5->gradio) (4.67.1)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.33.5->gradio) (1.1.3)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas<3.0,>=1.0->gradio) (2.9.0.post0)
```

```
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas<3.0,>=1.0->gradio) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas<3.0,>=1.0->gradio) (2025.2)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<2.12,>=2.0->gradio) (0.7.0)
Requirement already satisfied: pydantic-core==2.33.2 in /usr/local/lib/python3.12/dist-packages (from pydantic<2.12,>=2.0->gradio) (2.33.2)
Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<2.12,>=2.0->gradio) (0.4.2)
Requirement already satisfied: click>=8.0.0 in /usr/local/lib/python3.12/dist-packages (from typer<1.0,>=0.12->gradio) (8.3.0)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer<1.0,>=0.12->gradio) (1.5.4)
Requirement already satisfied: rich>=10.11.0 in /usr/local/lib/python3.12/dist-packages (from typer<1.0,>=0.12->gradio) (13.9.4)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas<3.0,>=1.0->gradio) (1.17.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=10.11.0->typer<1.0,>=0.12->gradio) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=10.11.0->typer<1.0,>=0.12->gradio) (2.19.1)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>huggingface-hub<2.0,>=0.33.5->gradio) (3.4.0)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>huggingface-hub<2.0,>=0.33.5->gradio) (2.3.0)
Requirement already satisfied: mdurl==0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=10.11.0->typer<1.0,>=0.12->gradio) (0.1.2)
```



```

import gradio as gr
import tensorflow as tf
import numpy as np

# Prediction function
def predict(image):
    # Resize and normalize image
    img = tf.image.resize(image, (128, 128)) # match model input size
    img = tf.expand_dims(img, axis=0)       # add batch dimension

    # Predict
    pred = model.predict(img)
    class_idx = np.argmax(pred[0])
    class_name = class_names[class_idx]
    confidence = float(pred[0][class_idx])

    return {class_name: confidence}

```

```

# Gradio interface
interface = gr.Interface(
    fn=predict,
    inputs=gr.Image(type="numpy"),
    outputs=gr.Label(num_top_classes=3),
    title="Vehicle Classification",
    description="Upload a vehicle image to classify it."
)

# Launch the app
interface.launch()

```

It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`. Automatically setting `share=True` (you can

Colab notebook detected. To show errors in colab notebook, set debug=True in launch())

* Running on public URL: <https://87e153a7bcf5b1ea96.gradio.live>

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working directo

Vehicle Classification

Upload a vehicle image to classify it.

🖼️
image

Drop Image Here
- or -
Click to Upload

📁
🔍
🗑️

📄
output

Flag

Clear

Submit

Vehicle Classification

Upload a vehicle image to classify it.

image





output

Bus

Bus

100%

Flag

Clear

Submit

Vehicle Classification

Upload a vehicle image to classify it.

image





output

Motorcycles

Motorcycles

100%

Flag

Clear

Submit

Vehicle Classification

Upload a vehicle image to classify it.

image





output

Ships

Ships

100%

Flag

Clear

Submit

Vehicle Classification

Upload a vehicle image to classify it.

image



✕

output

Planes

Planes 100%

Flag

Clear

Submit

1. Abstract

Vehicle classification plays a crucial role in intelligent transportation systems (ITS), automated surveillance, and traffic analytics. Convolutional Neural Networks (CNNs) have shown remarkable performance in complex image identification tasks due to the quick development of deep learning. The goal of this project is to create a CNN-based multi-class vehicle type classification model that can identify four different types of vehicles: **buses, motorcycles, planes and ships**. Because there were no typical train/validation splits, a custom dataset had to be gathered and organized manually. The dataset was preprocessed, standardized, and split into 80% training and 20% validation data using Google Colab.

Multiple convolutional, pooling, and fully connected layers were used in the CNN model's construction. Model training, assessment, and performance visualization came next. To evaluate the effectiveness of the model, confusion matrices, accuracy and loss curves, and classification metrics were created. Gradio was used for developing a graphical user interface (GUI) that enables real-time vehicle predictions using user-uploaded photos.

The study effectively illustrates the potential of CNNs in practical vehicle recognition systems by demonstrating an end-to-end pipeline of dataset preparation, deep learning model building, and practical deployment.

2. Introduction

Automatic vehicle type classification is now much more necessary due to the quick growth of smart city projects and the incorporation of intelligent transportation systems. Traffic management, road safety, toll monitoring, number plate recognition pipelines, congestion analysis, and autonomous driving modules all depend on accurate vehicle identification. Feature extraction in traditional computer vision required manual engineering, which is frequently inconsistent because of changes in environmental factors including lighting, angle, motion blur, occlusion, and background clutter.

Deep learning, particularly Convolutional Neural Networks (CNNs), has revolutionized this field by enabling models to learn complex spatial features automatically. CNNs are appropriate for vehicle classification applications because of their exceptional performance in object detection, facial recognition, and medical picture analysis.

The use of CNNs to categorize different vehicles photos into predefined groups is investigated in this study. Because publicly accessible datasets either needed preprocessing or included unnecessary vehicle kinds, a customized dataset was created. The dataset consists of four major vehicle classes that commonly appear in real-world road scenarios.

The main goal of this work is to create a straightforward but efficient model that can accurately classify various vehicle types and capture their major visual characteristics. This project is appropriate for academic, research, and practical demonstration applications since it incorporates a Gradio-based graphical user interface (GUI) that makes it easy for end users to interact with the model.

3. Objectives

The primary objectives of the study are as follows:

3.1 Technical Objectives

- To gather and organize a unique dataset that includes a variety of vehicle photos.
- To implement an automated train–validation splitting mechanism for balanced learning.
- To design and build robust CNN architecture capable of multi-class vehicle classification.
- To enhance model generalization by preprocessing, normalizing, and augmenting data.
- To monitor model training progress using accuracy and loss visualizations.
- To evaluate the model using classification reports and confusion matrices.

3.2 Real-World Objectives (Future Work)

- To investigate how deep learning might be used in intelligent transportation systems (ITS).
- To create a straightforward deployment interface that would allow end users to test the model in real-world scenarios.
- To identify limitations and propose improvements for future research.

4. Dataset Description

A custom dataset titled vehicle-dataset was developed for this project. The dataset contains four folders, each representing a separate vehicle category:

- Planes
- Bus
- Ships
- Motorcycle

4.1 Data Source

The images were collected from open-source platforms such as:

- Google Images
- Kaggle datasets

4.2 Dataset Characteristics

- Mixed background environments
- Varying vehicle angles (front, side, rear)
- Multiple lighting conditions
- Real-world road scenarios

4.3 Dataset Splitting

Since the dataset did not include predefined training or validation data, an automated Python script was applied to split the dataset into:

- 80% Training Set
- 20% Validation Set

This ensured balanced representation across all four classes, preventing data leakage.

5. Methodology

The methodology describes the complete workflow:

5.1 Data Upload & Directory Preparation

The dataset was uploaded into Google Colab and extracted into the working directory. After extraction, a train–validation split was performed.

5.2 Data Preprocessing

- All images were resized to 128×128 px and kernel and pool size 3×3 px.
- Shuffling ensured random batch creation.
- RGB normalization standardized pixel values.
- Dataset was loaded using TensorFlow's `image_dataset_from_directory()`.

5.3 Data Augmentation

- Data augmentation increased training data variability.
- Rotation, zoom, and flipping were applied to images.
- Augmented images preserved important vehicle features.
- Helped reduce overfitting during training.
- Improved model generalization and accuracy.

5.4 CNN Architecture

The CNN model included:

- Conv2D layers to extract spatial features.
- Batch Normalization for stable learning.
- MaxPooling layers for dimensionality reduction.
- Flatten layer to convert feature maps into vectors.
- Dense layers for classification.
- Dropout layers to prevent overfitting.
- Softmax output layer for four-class predictions.

The model was compiled using:

- **Optimizer:** Adam
- **Loss Function:** Sparse Categorical Crossentropy
- **Evaluation Metric:** Accuracy

5.5 Model Training

Training was conducted for 20 epochs, and colab's GPU support accelerated computation. Learning tendencies were observed by plotting accuracy and loss graphs.

5.6 Model Evaluation

Performance evaluation included:

- Validation accuracy
- Confusion matrix
- Precision, recall, F1-score
- Execution time of inference
- ROC Curve
 - Shows class-wise performance
 - Independent of classification threshold
 - More informative than accuracy alone

5.7 GUI Integration

To demonstrate real-world usability, the model was deployed using a Gradio interface where users can upload any image to receive vehicle type predictions.

6. Results & Analysis

The trained model was evaluated on the validation dataset, yielding moderate performance.

6.1 Training Performance

The accuracy curve shows:

- Rapid improvement during initial epochs
- Slower convergence towards later epochs
- Minor gap between training and validation accuracy

The loss curve indicates:

- Gradual reduction in training loss
- A slightly higher validation loss indicating mild overfitting

6.2 Classification Metrics

The classification report highlighted:

- The model achieved almost an overall accuracy of 90%, showing strong performance across all four classes.
- Motorcycles and Planes showed the highest recall due to their distinct visual features.
- Bus and Ships had slightly lower scores, mainly because of variations in size, background, and shape.
- Both macro and weighted averages scored 0.89, indicating balanced performance across classes.
- The confusion matrix shows most predictions are correct, with minor misclassifications between visually similar categories.

6.3 Confusion Matrix Interpretation

The confusion matrix indicated:

- Most predictions were correct
- Some classes overlapped due to dataset similarities

6.4 Validation Accuracy

Final validation accuracy ranged between 85–90%, depending on dataset complexity and training conditions.

This accuracy is acceptable for a custom dataset without augmentation.

7. Conclusion

In this project, a functional vehicle type classification system was developed using Convolutional Neural Networks. Every step of the process, from dataset development to GUI deployment, demonstrates how deep learning may be used to solve practical image identification issues. Using Gradio, the system effectively divides vehicles photos into four main categories and makes predictions in real time.

The study also highlights several future improvements:

- Using transfer learning to enhance accuracy
- Increasing dataset size
- Applying augmentation techniques more precisely
- Deploying the model as a web or mobile application

Overall, the project demonstrates a complete and practical implementation of CNNs, which is suitable for real-world use.